



딥러닝 파이토치 5장(Part 5.1, 5.2): 합성곱 신경망 I

⚙ Status	In progress
👤 Assignee	 시현 이
📅 Due date	@03/24/2026
🔧 Task type	

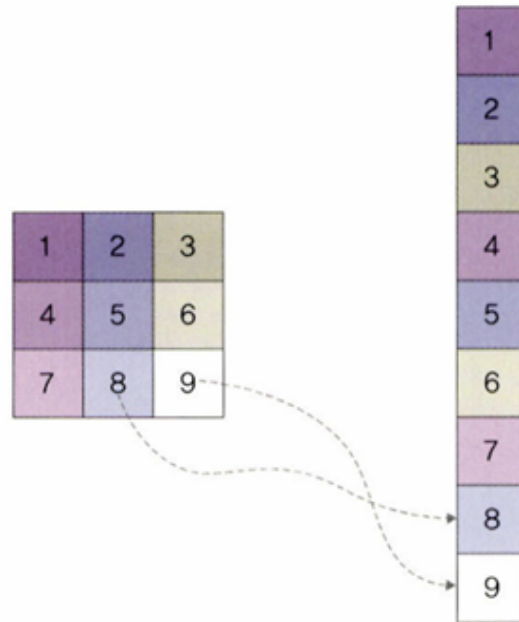
5.1 합성곱 신경망

딥러닝의 순전파, 역전파 계산 과정은 복잡하고 많은 자원을 요구함 + 계산 시간 오래걸림

⇒ 해결방법: 이미지의 국소 부위를 계산해 시간과 자원을 절약하며 세밀하게 분석할 수 있는
신경망인 '합성곱 신경망'

5.1.1 합성곱층의 필요성

▼ 그림 5-1 합성곱층 원리



3*3 흑백 이미지가 다음과 같이 존재한다고 가정해보자.

이미지 분석은 다음 그림을 왼쪽과 같이 펼쳐서(flattening) 각 픽셀에 가중치를 곱해 은닉층으로 전달한다.

⇒ 이미지를 펼치는 것은 데이터의 공간적 구조를 무시하게 되는데, 이를 방지하기 위해 도입된 것이 합성곱층이다.

5.1.2 합성곱 신경망 구조

합성곱 신경망(CNN 또는 ConvNet)

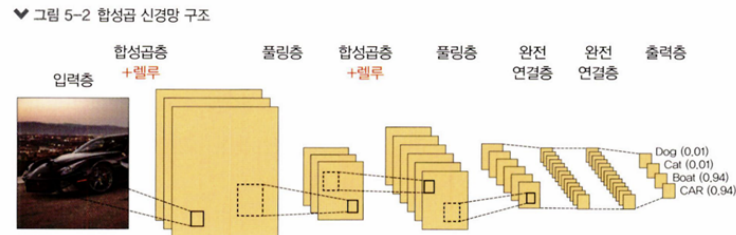
: 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망

: 다차원 배열 데이터를 처리하도록 구성돼 있어 컬러 이미지 같은 다차원 배열 처리에 특화

[구성요소]

- 입력층
- 합성곱층
- 풀링층

- 완전연결층
- 출력층



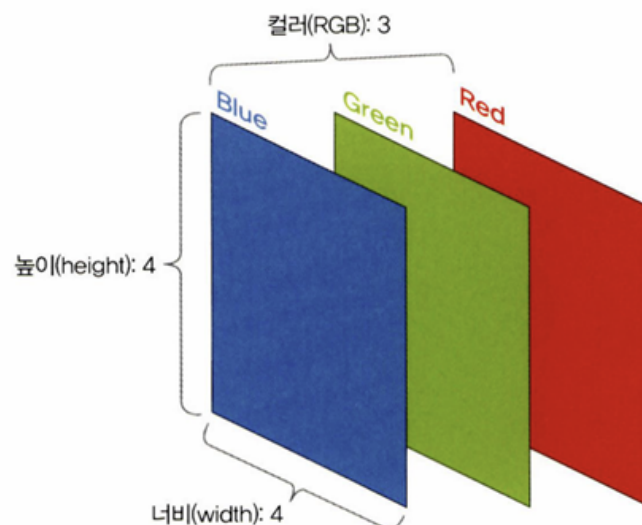
- ⇒ 합성곱층과 풀링층을 거치며 입력 이미지의 주요 특성 벡터를 추출
 → 추출된 주요 특성 벡터들은 완전연결층을 거치며 1차원 벡터로 변환
 → 출력층에서 활성화함수인 소프트맥스 함수를 사용해 최종 결과가 출력됨

입력층(input layer)

: 입력 이미지 데이터가 최초로 거치게 되는 계층

- 높이, 너비, 채널의 값을 갖는 3차원 데이터
- 채널은 이미지가 그레이스케일이면 == 1, 컬러면(RGB) == 3
- 다음은 이미지 형태가 (4,4,3)

▼ 그림 5-3 채널



합성곱층(convolutional layer)

: 입력 데이터에서 특성을 추출하는 역할

- 특성추출
 - 입력 이미지가 들어왔을 때 이미지에 대한 특성을 감지하기 위해 커널이나 필터를 사용
 - 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출 ⇒ 이렇게 추출한 결과물: '**특성 맵(feature map)**'
 - 커널은 3*3, 5*5 크기로 적용되는 것이 일반적
 - stride라는 지정된 간격에 따라 순차적으로 이동

[그레이스케일]

다음은 (6,6,1)인 이미지에 3*3 크기의 커널/필터를 스트라이드 1만큼 이동하며 합성곱 연산 수행하는 과정

1단계. 입력 이미지에 3*3 필터 적용

입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더함

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

▼ 그림 5-4 입력 이미지에 3*3 필터 적용



2단계. 필터가 1만큼 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



3단계. 필터가 1만큼 두번째 이동

$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

▼ 그림 5-6 입력 이미지에 필터가 1만큼 두 번째 이동



4단계. 필터가 1만큼 세번째 이동

$$(0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 3$$

▼ 그림 5-7 입력 이미지에 필터가 1만큼 세 번째 이동



5단계. 필터가 1만큼 네번째 이동

$$(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) = 1$$

▼ 그림 5-8 입력 이미지에 필터가 1만큼 네 번째 이동



6단계. 필터가 1만큼 마지막으로 이동

$$(0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) = 1$$

▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동

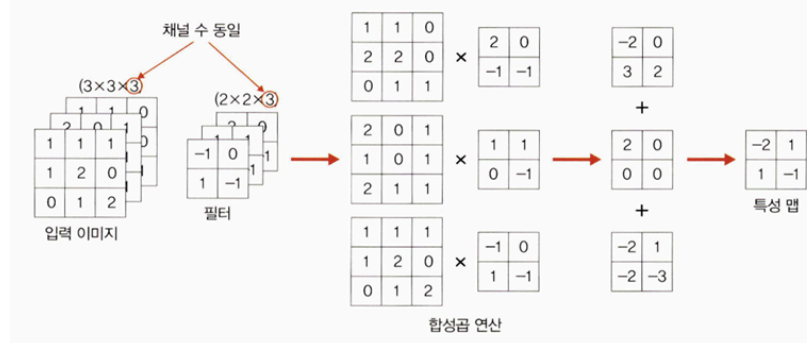


⇒ (4,4,1) 크기의 특성 맵으로 줄어들음

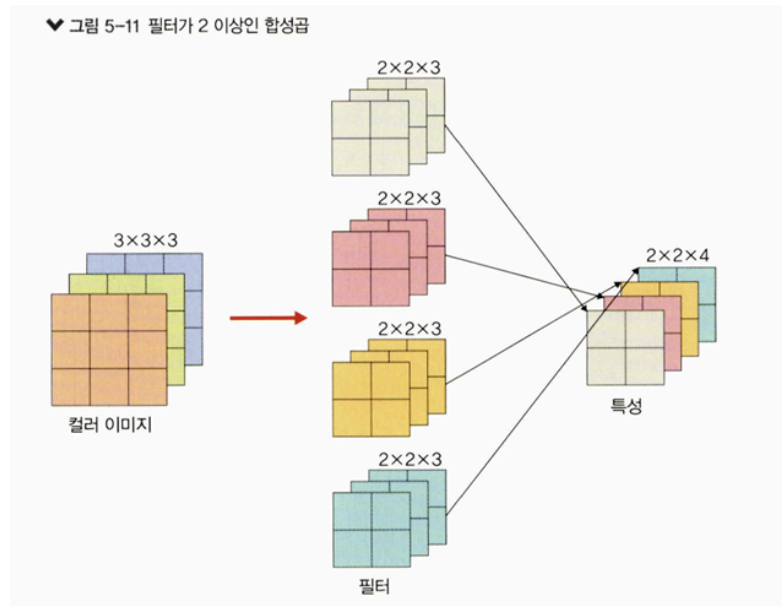
[컬러 이미지]

1. 필터 채널이 3(실제로는 필터 개수가 1개임)
2. RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌
3. 그 외는 스트라이드 및 연산하는 방법 동일

▼ 그림 5-10 컬러 이미지 합성곱



[필터가 2개 이상인 합성곱]



- 입력 데이터: $W_1 \times H_1 \times D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

풀링층(pooling layer)

: 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출해 학습을 효과적으로 할 수 있게 함



다운 샘플링(sub-sampling)

:이미지 축소



풀링 연산

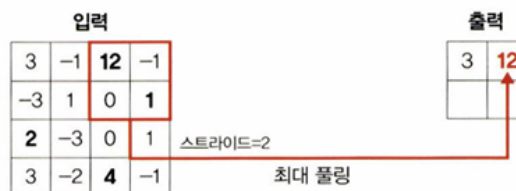
- **최대 풀링(max pooling):** 대상 영역에서 최댓값을 추출
 - 대부분의 합성곱 신경망에서는 최대 풀링 사용
 - 최대 풀링 연산과정
 - 첫번째 최대풀링 과정(최댓값 선택)

▼ 그림 5-13 첫 번째 최대 풀링 과정



- 두번째 최대풀링 과정

▼ 그림 5-14 두 번째 최대 풀링 과정



- 세번째 최대풀링 과정

▼ 그림 5-15 세 번째 최대 풀링 과정



■ 네번째 최대풀링 과정

▼ 그림 5-16 네 번째 최대 풀링 과정



- **평균 풀링(average pooling):** 대상 영역에서 평균을 반환
 - 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있음
 - 각 필터의 평균으로 계산

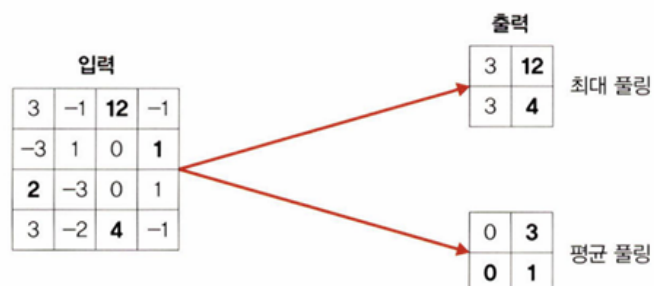
$$0 = (3 + (-1) + (-3) + 1) / 4$$

$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

▼ 그림 5-17 최대 풀링과 평균 풀링 비교

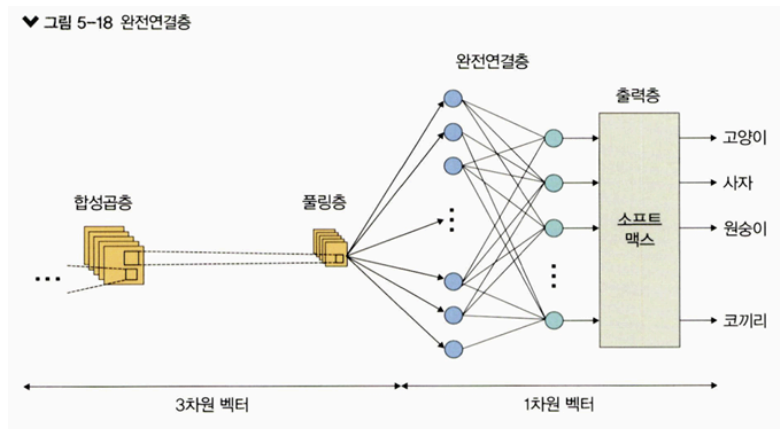


- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

완전연결층(fully connected layer)

: 합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전연결층으로 전달됨

: 이미지가 3차원 벡터에서 1차원 벡터로 펼쳐짐



출력층(output layer)

- 소프트맥스 활성화 함수 사용
- 입력 받은 값을 0~1 사이의 값으로 출력
- 출력층의 소프트맥스 함수를 사용해 이미지가 각 레이블에 속할 확률 값이 출력됨
 - 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정됨

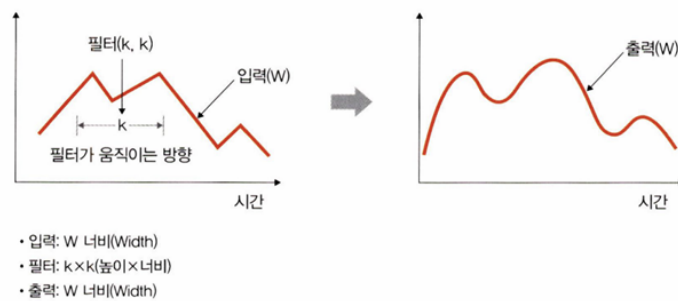
5.1.3 1D, 2D, 3D 합성곱

1D 합성곱

:필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱

- 입력(W)과 필터(k)에 대한 출력은 W
 - 예) dlqfurdl [1,1,1,1]이고 필터가 [0.25, 0.5, 0.25]라면 $\Rightarrow$ 출력은 [1,1,1]
 - 그래프 곡선을 완화할 때 많이 사용됨

♥ 그림 5-19 1D 합성곱

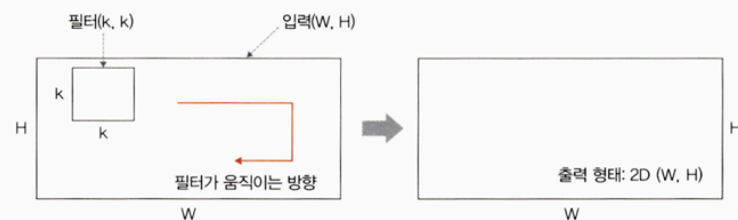


2D 합성곱

: 방향 2개로 움직이는 형태

- 입력(W,H)과 필터(k,k)에 대한 출력은 (W,H)가 됨 $\Rightarrow$ 출력 형태는 2D 행렬

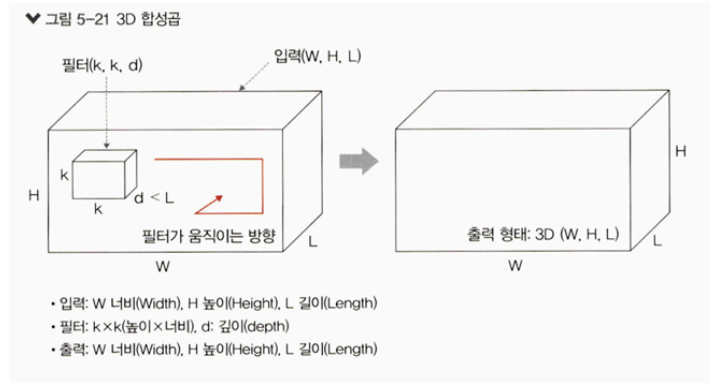
♥ 그림 5-20 2D 합성곱



3D 합성곱

: 필터가 움직이는 방향이 3개

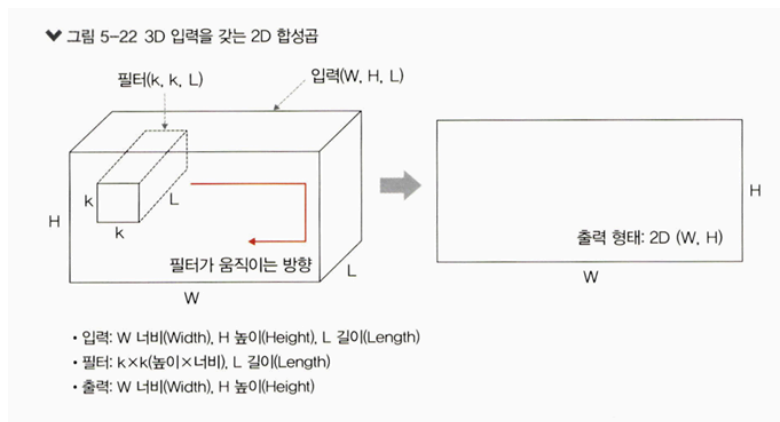
- 입력(W,H,L)에 대해 필터(k,k,d)를 적용하면 $\Rightarrow$ 출력으로 (W,H,L)을 갖는 형태가 3D 합성곱
 - 이때, $d < L$ 을 유지하는게 중요



3D 입력을 갖는 2D 합성곱

: 입력이 3D 형태임에도 출력 형태가 2D 행렬을 취하는 것

- 필터에 대한 길이(L)이 입력 채널의 길이(L)와 같아야함
 - 입력(W,H,L) + 필터(k,k,L) $\Rightarrow$ 출력(W,H)
- 대표적 사례: LeNet-5, VGG

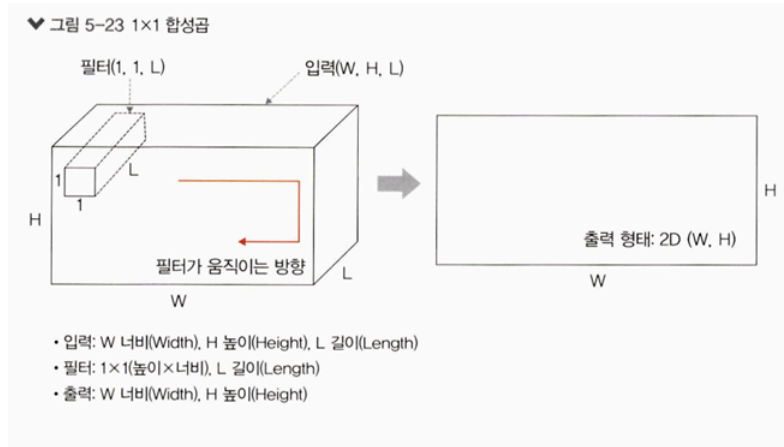


1*1 합성곱

: 3D 형태로 입력됨

- 입력(W,H,L) + 필터(1,1,L) $\Rightarrow$ 출력(W,H)
- 1*1 합성곱에서 채널 수를 조정해 연산량이 감소되는 효과

- 대표적 사례: GoogLeNet



5.2 합성곱 신경망 맛보기



fashion_mnist 데이터셋

- torchvision에 내장된 예제 데이터로 운동화, 셔츠, 샌들 같은 작은 이미지의 모음
- 기본 MNIST 데이터셋처럼 열 가지로 분류될 수 있는 28*28 픽셀의 이미지 7만 개로 구성됨
- 훈련 데이터(train_images): 0~255 사이의 값을 갖는 28*28 크기의 넘파이 배열
- 레이블(정답) 데이터(train_labels): 0~9까지의 정수 값을 갖는 배열
 - 0~9 정수 값: 이미지의 클래스를 나타내는 레이블

```
0 : T-Shirt
1 : Trouser
2 : Pullover
3 : Dress
4 : Coat
5 : Sandal
6 : Shirt
7 : Sneaker
8 : Bag
9 : Ankle Boot
```

Supporting files

https://colab.research.google.com/drive/1_cnutiuMWpIVSIQp87D6RgBBNQ_c61n_